

Basic C Without Tears

Guía Práctica con Soluciones – Tipo B

Estructuras, Punteros, Memoria Dinámica y Operaciones de Bits

Autor: Stack

1. Responda a las siguientes preguntas sobre estructuras, uniones y representación de datos compuestos en memoria.

- (a) Considere las siguientes dos definiciones en una arquitectura de 64 bits con alineación natural ($\text{sizeof}(\text{char}) = 1$, $\text{sizeof}(\text{int}) = 4$, $\text{sizeof}(\text{double}) = 8$):

```
struct Version_A {
    char    tipo;
    double  valor;
    int     codigo;
};

struct Version_B {
    double  valor;
    int     codigo;
    char    tipo;
};
```

Sin modificar los campos, indique cuál de las dos versiones ocupa menos bytes en memoria y justifique calculando el `sizeof` de cada una. Detalle los bytes de *padding* insertados y su ubicación.

Solución y Explicación:

Respuesta: Version_B ocupa menos memoria (**16 bytes**) frente a los **24 bytes** de Version_A.

Análisis de Version_A (24 bytes):

- Offset 0: char tipo (1 byte) + **7 bytes de padding** (offsets 1–7) para alinear double a múltiplo de 8.
- Offsets 8–15: double valor (8 bytes).
- Offsets 16–19: int codigo (4 bytes) + **4 bytes de padding** (offsets 20–23) para que el tamaño total sea múltiplo de 8.

Análisis de Version_B (16 bytes):

- Offsets 0–7: double valor (8 bytes), ya alineado.
- Offsets 8–11: int codigo (4 bytes), alineado a múltiplo de 4.
- Offset 12: char tipo (1 byte) + **3 bytes de padding** (offsets 13–15).

Reordenar los campos de mayor a menor tamaño minimiza el padding.

- (b) El siguiente programa intenta copiar el contenido de una estructura a otra, pero el programador usó asignación campo a campo. Determine qué imprime el código y explique por qué la asignación directa `b = a;` entre variables del mismo tipo `struct` es válida en C:

```

1 struct Punto { int x; int y; };
2
3 struct Punto a = {15, -8};
4 struct Punto b;
5 b.x = a.x;
6 b.y = a.y;
7 a.x = 0;
8
9 printf("b: (%d, %d)\n", b.x, b.y);

```

Solución y Explicación:

Salida por pantalla: b: (15, -8)

La asignación campo a campo copia los valores de `a` en `b`. Ambas variables son independientes en el Stack. Cuando `a.x = 0;` se ejecuta después, solo modifica la copia de `a`, sin afectar a `b` que ya almacena 15.

Sobre `b = a;`: En C, la asignación directa entre variables del mismo tipo `struct` es válida y equivale a una copia byte a byte (`memcpy` implícito) de todos los campos. El compilador genera el código de copia automáticamente.

- (c) Escriba una función en C con el siguiente prototipo:

```
int contar_aprobados(struct Alumno lista[], int n);
```

donde `struct Alumno` contiene los campos `char nombre[50]` y `float promedio`. La función debe retornar la cantidad de alumnos cuyo promedio sea mayor o igual a 4.0.

Entrada	Salida esperada
{("Ana", 5.2), ("Luis", 3.9), ("Mia", 4.0)}	2
{("Pedro", 3.5), ("Sol", 2.8)}	0

Solución y Explicación:

```

1 struct Alumno {
2     char nombre[50];
3     float promedio;
4 };
5
6 int contar_aprobados(struct Alumno lista[], int n) {
7     int aprobados = 0;
8     for (int i = 0; i < n; i++) {
9         if (lista[i].promedio >= 4.0f) {
10             aprobados++;
11         }
12     }
13     return aprobados;
14 }

```

La función recorre el arreglo accediendo al campo `promedio` con el operador punto (`.`). La comparación `>= 4.0f` incluye igualdad, por lo que un promedio de exactamente 4.0 cuenta como aprobado.

2. Los punteros en C permiten manipular directamente las direcciones de memoria donde residen las variables. Cada tipo de puntero avanza en memoria según el tamaño del tipo al que apunta.

- (a) Determine qué imprime el siguiente fragmento de código. Justifique cada línea de salida indicando el valor almacenado y la operación realizada:

```
1 int datos[5] = {100, 200, 300, 400, 500};
2 int *p = datos + 1;
3
4 printf("%d\n", *p);
5 printf("%d\n", *(p + 2));
6 printf("%d\n", p[-1]);
7 p += 3;
8 printf("%d\n", *p);
```

Solución y Explicación:

El puntero `p` se inicializa apuntando a `datos[1]`:

- 1) `*p`: Desreferencia la posición actual $\rightarrow$ `datos[1] = 200`.
- 2) `*(p + 2)`: Avanza 2 posiciones desde `datos[1]` $\rightarrow$ `datos[3] = 400`.
- 3) `p[-1]`: Retrocede 1 posición desde `datos[1]` $\rightarrow$ `datos[0] = 100`.
- 4) Tras `p += 3`, `p` apunta a `datos[4]`. `*p = 500`.

Salida: 200, 400, 100, 500 (una por línea).

- (b) El siguiente programa pretende intercambiar los valores de dos variables mediante una función, pero tras la llamada `x` e `y` permanecen inalteradas:

```
1 void intercambiar(int a, int b) {
2     int temp = a;
3     a = b;
4     b = temp;
5 }
6
7 int main() {
8     int x = 7, y = 13;
9     intercambiar(x, y);
10    printf("x=%d, y=%d\n", x, y);
11    return 0;
12 }
```

Explique por qué falla el intercambio y reescriba la función corregida utilizando punteros.

Solución y Explicación:

Diagnóstico: En C, los argumentos se pasan **por valor**. Las variables `a` y `b` son copias locales que se destruyen al retornar. Salida: `x=7, y=13`.

Corrección con punteros:

```
1 void intercambiar(int *a, int *b) {
2     int temp = *a;
3     *a = *b;
4     *b = temp;
5 }
6 // Llamada: intercambiar(&x, &y);
```

(c) Escriba una función en C con el siguiente prototipo:

```
void separar_pares_impares(const int *origen, int n,  
    int *pares, int *np, int *impares, int *ni);
```

La función recorre el arreglo `origen` de n elementos y copia los números pares en `pares` (cantidad en `*np`) y los impares en `impares` (cantidad en `*ni`).

Entrada (origen)	Salida esperada
{3, 8, 15, 4, 7, 2}	pares: 8 4 2 (3), impares: 3 15 7 (3)
{10, 20, 30}	pares: 10 20 30 (3), impares: (0)

Solución y Explicación:

```
1 void separar_pares_impares(const int *origen, int n,  
2     int *pares, int *np, int *impares, int *ni) {  
3     *np = 0;  
4     *ni = 0;  
5  
6     for (int i = 0; i < n; i++) {  
7         if (origen[i] % 2 == 0) {  
8             pares[*np] = origen[i];  
9             (*np)++;  
10        } else {  
11            impares[*ni] = origen[i];  
12            (*ni)++;  
13        }  
14    }  
15 }
```

Puntos clave:

- Los contadores `*np` y `*ni` se inicializan a 0 al inicio.
- Se usa el operador módulo (`% 2`) para clasificar cada elemento.
- Los paréntesis en `(*np)++` son esenciales: sin ellos, la precedencia de `++` incrementaría el puntero en lugar del valor apuntado.
- `const` en `origen` garantiza que no se modifica el arreglo de entrada.

3. La gestión explícita de memoria dinámica en C se realiza mediante las funciones `malloc`, `realloc` y `free` de la biblioteca `<stdlib.h>`.

- (a) El siguiente código compila correctamente, pero contiene dos errores graves de gestión de memoria. Identifíquelos y explique sus consecuencias:

```
1 int *crear_secuencia(int n) {
2     int resultado[n];
3     for (int i = 0; i < n; i++) {
4         resultado[i] = (i + 1) * 10;
5     }
6     return resultado;
7 }
8
9 int main() {
10     int *sec = crear_secuencia(5);
11     for (int i = 0; i < 5; i++) {
12         printf("%d ", sec[i]);
13     }
14     free(sec);
15     return 0;
16 }
```

Solución y Explicación:

Error 1 – Retorno de puntero a variable local (*dangling pointer*): El arreglo `resultado` es un VLA que vive en el Stack Frame de `crear_secuencia`. Cuando la función retorna, su Stack Frame se destruye y la memoria queda invalidada. El puntero devuelto produce *undefined behavior* al desreferenciarlo.

Error 2 – `free` sobre memoria no dinámica: La función `free` solo acepta punteros obtenidos mediante `malloc`, `calloc` o `realloc`. Llamar a `free(sec)` con un puntero al Stack provoca corrupción del Heap o *Segmentation Fault*.

- (b) Reescriba la función `crear_secuencia` de forma correcta, reservando memoria en el Heap con `malloc` y verificando que la asignación no falle. Indique quién es responsable de liberar la memoria y dónde debe hacerse.

Solución y Explicación:

```
1 int *crear_secuencia(int n) {
2     int *resultado = (int *)malloc(n * sizeof(int));
3     if (resultado == NULL) {
4         return NULL; // Fallo de asignacion
5     }
6     for (int i = 0; i < n; i++) {
7         resultado[i] = (i + 1) * 10;
8     }
9     return resultado;
10 }
```

Responsabilidad de liberar: El llamador (`main`) debe invocar `free(sec)` cuando termine de usar el bloque. La memoria en el Heap persiste hasta que se libere explícitamente.

(c) Escriba una función en C con el siguiente prototipo:

```
int *leer_hasta_cero(int *cantidad);
```

La función lee enteros desde la entrada estándar hasta que el usuario ingrese el valor 0 (centinela). Debe almacenar los valores leídos en un bloque dinámico que comienza con capacidad para 2 elementos y se duplica con `realloc` cada vez que se llena. Al finalizar, almacena en `*cantidad` el total de elementos leídos y retorna el puntero al bloque.

Solución y Explicación:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int *leer_hasta_cero(int *cantidad) {
5     int capacidad = 2;
6     int total = 0;
7     int *bloque = (int *)malloc(capacidad * sizeof(int));
8     if (bloque == NULL) {
9         *cantidad = 0;
10        return NULL;
11    }
12
13    int valor;
14    while (scanf("%d", &valor) == 1 && valor != 0) {
15        if (total == capacidad) {
16            capacidad *= 2;
17            int *temp = (int *)realloc(bloque,
18                                     capacidad * sizeof(int));
19            if (temp == NULL) {
20                free(bloque);
21                *cantidad = 0;
22                return NULL;
23            }
24            bloque = temp;
25        }
26        bloque[total] = valor;
27        total++;
28    }
29
30    *cantidad = total;
31    return bloque;
32 }
```

Puntos clave:

- Se usa un puntero temporal `temp` para el `realloc`: si falla, se conserva el puntero original para liberarlo y evitar una fuga de memoria.
- La capacidad se duplica ($\times 2$) en cada expansión, dando un crecimiento amortizado de $O(1)$ por inserción.
- El centinela (0) no se almacena en el arreglo.

4. Las operaciones a nivel de bits (&, |, ^, ~, «, ») permiten manipular individualmente los bits de un valor entero.

(a) Determine qué imprime el siguiente fragmento de código. Muestre el cálculo binario completo de cada operación (use representación de 8 bits):

```
1 unsigned char a = 0xC5; // 1100 0101
2 unsigned char b = 0x3A; // 0011 1010
3
4 printf("AND: 0x%02X\n", a & b);
5 printf("OR: 0x%02X\n", a | b);
6 printf("XOR: 0x%02X\n", a ^ b);
```

Solución y Explicación:

Operandos: a = 0xC5 = 1100 0101, b = 0x3A = 0011 1010.

- **AND** (a & b): 1100 0101 & 0011 1010 = 0000 0000 = **0x00**.
- **OR** (a | b): 1100 0101 | 0011 1010 = 1111 1111 = **0xFF**.
- **XOR** (a ^ b): 1100 0101 ^ 0011 1010 = 1111 1111 = **0xFF**.

Salida: AND: 0x00, OR: 0xFF, XOR: 0xFF.

(b) En un sistema de control de acceso, los permisos de un usuario se almacenan en una variable `unsigned char permisos` donde cada bit representa un permiso distinto:

- Bit 0: Lectura Bit 1: Escritura Bit 2: Ejecución Bit 3: Administración

Un usuario tiene `permisos = 0x05` (binario: 0000 0101). Escriba las expresiones en C para:

1. Verificar si tiene permiso de Escritura (bit 1).
2. Otorgarle permiso de Administración (bit 3) sin alterar los demás.
3. Revocarle permiso de Lectura (bit 0) sin alterar los demás.
4. Conmutar (toggle) el permiso de Ejecución (bit 2).

Solución y Explicación:

Estado inicial: `permisos = 0x05 = 0000 0101` (Lectura + Ejecución activos).

1) **Verificar Escritura (bit 1):**

```
if (permisos & (1 << 1)) // 0x05 & 0x02 = 0x00 -> NO tiene
```

2) **Otorgar Administración (bit 3) – SET con OR:**

```
permisos |= (1 << 3); // 0x05 | 0x08 = 0x0D
```

3) **Revocar Lectura (bit 0) – CLEAR con AND + NOT:**

```
permisos &= ~(1 << 0); // 0x0D & 0xFE = 0x0C
```

4) **Conmutar Ejecución (bit 2) – TOGGLE con XOR:**

```
permisos ^= (1 << 2); // 0x0C ^ 0x04 = 0x08
```